CS 301 · MEETING 13 · A DEEPER LOOK

Elimination and Fill-in

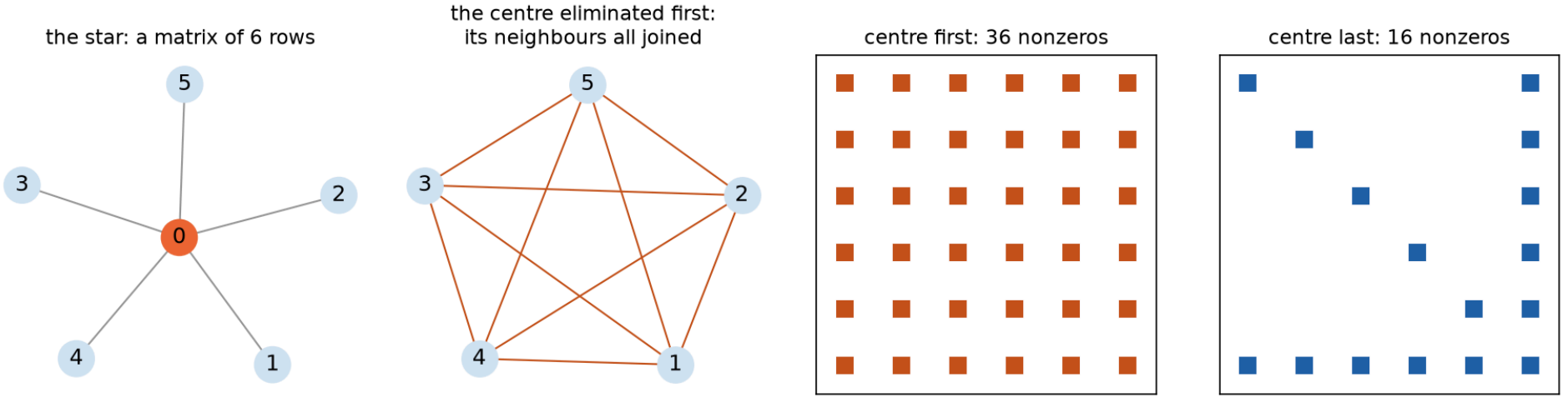
How elimination creates nonzeros and how ordering affects memory use and running time.

Elimination creates nonzeros

$$A[i, j] - \ = \text{factor} \cdot A[k, j] \quad \text{for every } j \geq k$$

- Row updates can turn zeros into nonzeros: **fill-in**
- The factors can be much denser than A
- The elimination order affects the amount of fill-in

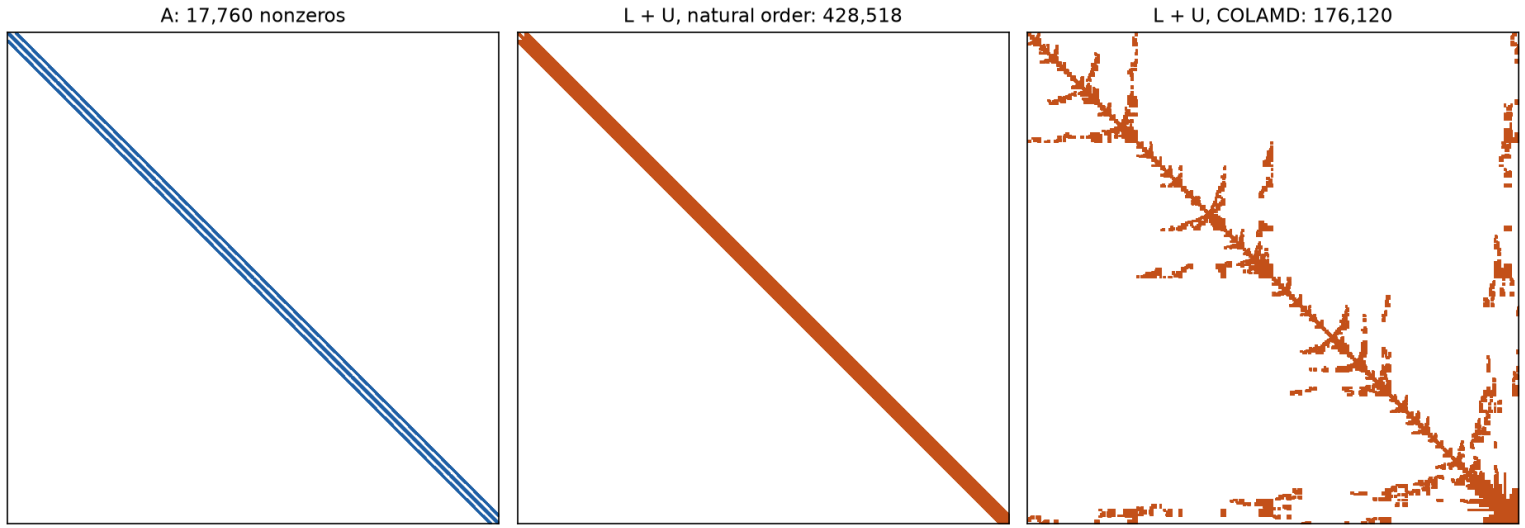
On the graph of the matrix




```
nonzeros of the star matrix: 16;  after elimination, centre first: 36  centre last: 16
```

- Symmetric A: off-diagonal nonzeros connect unknowns
- Eliminating a node connects its remaining neighbours
- Centre first: all leaves joined; centre last: no fill-in

The grid, eliminated in two orders



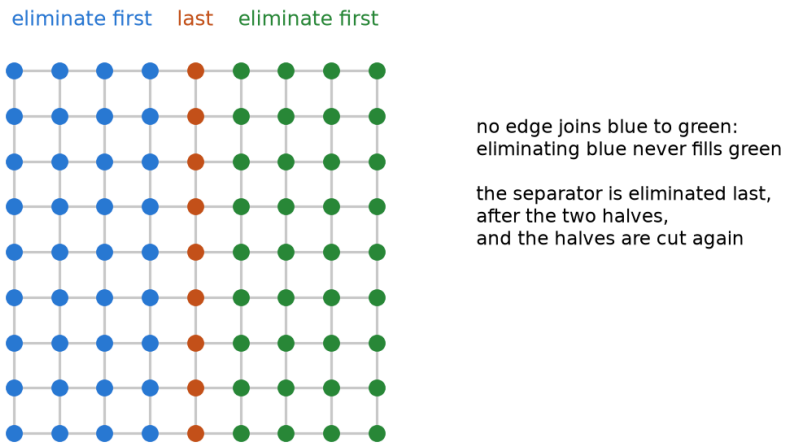

```
natural order                    428,518    24.1
COLAMD (SuperLU's default)       176,120     9.9
minimum degree                   104,830     5.9
```

Fill-in and solve time

grid	N	order	nonzeros of L + U	seconds to solve
80 x 80	6,400	natural	1,024,158	0.033
80 x 80	6,400	COLAMD	370,282	0.009
160 x 160	25,600	natural	8,192,318	0.410
160 x 160	25,600	COLAMD	2,036,562	0.053

- Compare factor nonzeros and solve time for each order
- `spsolve` applies a fill-reducing ordering by default

Nested dissection, in one picture



- Cut the grid by a **separator**; eliminate the two halves first, the separator last
- No edge joins the halves, so eliminating one never fills the other
- Cut each half again: for a grid, O(N log N) nonzeros, O(N^1.5) time

Ordering and the cost of sparse elimination

- Sparse elimination can create many additional nonzeros
- Eliminating a node connects its remaining neighbours
- Fill-reducing orderings limit factor storage and arithmetic work
- Nested dissection uses separators to control fill-in